Dynamic GraphRAG for Open Scientific Question Answering

A Graph-Augmented Retrieval System with Confidence-Driven Adaptive Expansion and Plan-Aware Pipeline Orchestration

Fadel Fatima zahra - Benfellah ikram - Samia lachgar

April 2026

Abstract

We present Dynamic GraphRAG, a multi-stage retrieval-augmented generation system that answers open-ended scientific questions by constructing ephemeral knowledge graphs from live academic literature. Unlike classical RAG pipelines that concatenate retrieved passages as flat text, our system extracts typed entities and directed relations from paper abstracts, builds a per-query NetworkX directed graph, retrieves a query-relevant subgraph via embedding-driven seed selection and multi-hop traversal, and conditions the final LLM answer on structured graph triples together with paper evidence. The pipeline is fully domain-agnostic, supporting Agriculture, Geoscience, Computer Science, Supply Chain, and Environment queries without domain-specific prompts. A confidence-driven adaptive expansion loop re-retrieves papers and re-builds the graph when coverage validation fails, bounded to at most two iterations. An LLM-based query planner classifies each question into one of five reasoning types and produces a structured plan that shapes retrieval weighting, entity extraction priorities, graph validation thresholds, and answer structure. A four-mode ablation evaluation framework with 22 benchmark questions measures ROUGE, keyword coverage, semantic similarity, and LLM-as-judge scores across progressive feature configurations.

Contents

1 Introduction **4**
1.1 Limitations of Classical RAG 4
1.2 Motivation for GraphRAG 4

2 System Overview **4**

3 Architecture **5**
3.1 Query Planner (`query_planner.py`) 5
3.2 Domain Detector (`domain_detector.py`) 6
3.3 Paper Retriever (`paper_retriever.py`) 6
3.4 Paper Filter (`paper_filter.py`) 7
3.5 Entity Extractor (`entity_extractor.py`) 7

3.6 Graph Builder (graph_builder.py) . . . 8
3.7 Graph Retriever (graph_retriever.py) . . . 8
3.8 Adaptive Retry (adaptive_retry.py) . . . 9
3.9 Answer Generator (answer_generator.py) . . . 9
3.10 Answer Compiler (answer_compiler.py) . . . 10
3.11 Confidence Scorer (confidence_scorer.py) . . . 10
3.12 Session State (session_state.py) . . . 10

4 Key Design Decisions **11**
4.1 Dynamic Graphs vs. Static Knowledge Bases . . . 11
4.2 Domain-Agnostic Entity Vocabulary . . . 11
4.3 Confidence-Driven Adaptive Expansion . . . 11
4.4 Caching Strategy . . . 11

5 Implementation Details **12**
5.1 Libraries and Dependencies . . . 12
5.2 External APIs . . . 12
5.3 Concurrency . . . 12
5.4 Caching Mechanisms . . . 12

6 Challenges Faced **13**
6.1 API Rate Limiting and Transient Failures . . . 13
6.2 LLM JSON Parsing Failures . . . 13
6.3 Entity Duplication Across Papers . . . 13
6.4 Latency from Sequential API Calls . . . 13
6.5 Sparse Graphs for Niche Queries . . . 14

7 Mistakes and Lessons Learned **14**
7.1 Over-Reliance on LLM Output Structure . . . 14
7.2 Hardcoded Configuration Drift . . . 14
7.3 Aggressive Filtering Removing Good Papers . . . 14
7.4 Graph Density vs. Noise Tradeoff . . . 14

8 Final Pre-Submission Improvements **14**
8.1 Removed Unused Dependencies . . . 15
8.2 Fixed Hardcoded Model Names . . . 15
8.3 Fixed max_tokens Inconsistency . . . 15
8.4 Fixed Temperature Inconsistency . . . 15
8.5 Logging Improvements . . . 15
8.6 Dead Code Removal . . . 15

9 Evaluation **15**
9.1 Evaluation Framework . . . 15
9.2 Ablation Modes . . . 16
9.3 Metrics . . . 16

10 Limitations **16**

11 Future Work **17**

12 Conclusion **17**

1 Introduction

1.1 Limitations of Classical RAG

Retrieval-augmented generation in its simplest form retrieves the top-k documents for a query and concatenates their text into a flat prompt for a large language model. This approach suffers from several well-known limitations in the scientific domain. First, flat concatenation loses structural relationships between entities: if Paper A states that drought *causes* stomatal closure and Paper B states that stomatal closure *reduces* photosynthesis, a flat prompt does not make the causal chain explicit. Second, relevance ranking operates at the document level rather than the concept level, so important entities mentioned in lower-ranked papers may be lost. Third, classical RAG provides no mechanism to assess whether the retrieved evidence is sufficient to answer a particular type of question (e.g., a comparison question requires at least two contrasting concepts).

1.2 Motivation for GraphRAG

Graph-based RAG addresses these gaps by introducing an explicit knowledge structure between retrieval and generation. Entities and relations extracted from retrieved papers form a directed graph that encodes the relational fabric of the evidence. Subgraph retrieval then selects a query-relevant neighborhood, and the LLM receives structured triples rather than raw text. This enables multi-hop reasoning (traversing cause-effect chains), structural validation (checking whether required entity types are present), and confidence-driven expansion (re-retrieving when the graph is too sparse).

Our system, Dynamic GraphRAG, implements this paradigm end to end with eight pipeline stages, a session-aware caching layer, and a confidence-driven expansion loop, all orchestrated by an LLM-generated query plan.

2 System Overview

The pipeline processes a natural-language scientific question through eight sequential stages, each implemented as an independent Python module. The stages communicate through explicit data contracts: each stage receives typed inputs and returns typed outputs, with no hidden shared state. The full pipeline is orchestrated by the `run_pipeline()` function in `main.py`.

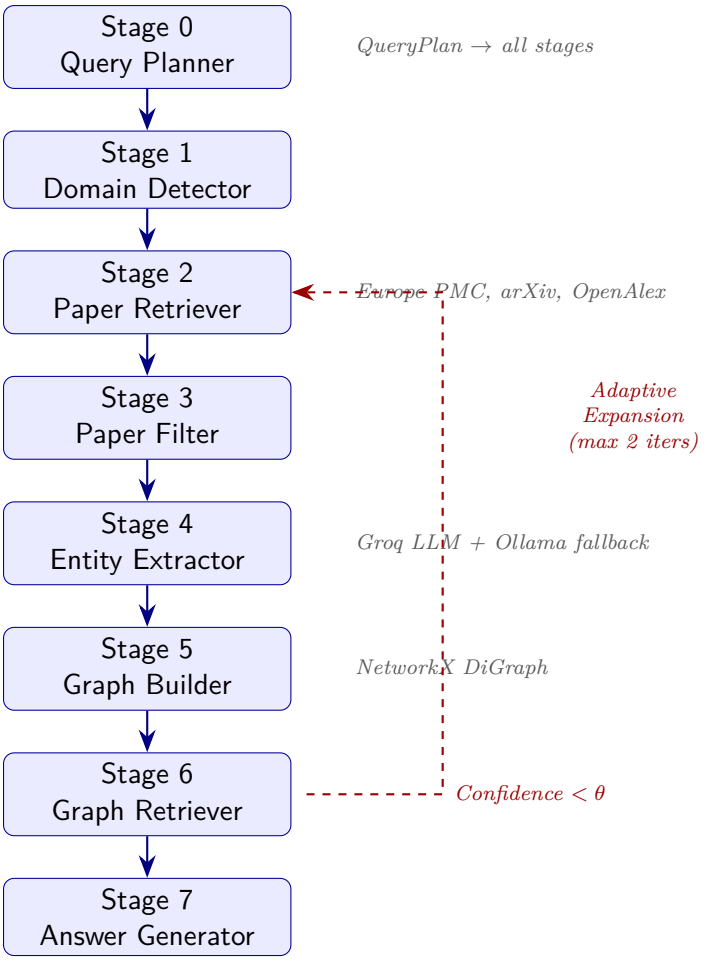

Figure 1: Dynamic GraphRAG pipeline with confidence-driven adaptive expansion loop.

The query plan produced in Stage 0 propagates to every downstream stage: it shapes retrieval weighting (Stage 2), entity extraction priorities (Stage 4), graph validation thresholds (Stage 6), and answer structure (Stage 7). If the confidence scorer determines that the graph is insufficient after Stage 6, the adaptive expansion loop re-enters at Stage 2 with an expanded query, merges the new evidence into the existing graph, and re-runs retrieval and answer generation.

3 Architecture

This section describes each implemented module in detail, based on the actual codebase.

3.1 Query Planner (`query_planner.py`)

The Query Planner is Stage 0 of the pipeline. It classifies each incoming question into one of five domain-agnostic *reasoning types*: definition, comparison, method/process, causal mechanism, or optimization. Classification is performed by an LLM (Groq API, model configurable via `QUERY_PLANNER_MODEL`) that returns a structured JSON object containing the reasoning type, key entities, comparison dimensions, intent signals, and complexity level.

Fallback mechanism. If the LLM call fails or returns unparseable JSON, a heuristic classifier activates. It matches the query against curated keyword patterns (e.g., "what is" → definition, "why" → causal_mechanism) and extracts noun-phrase entities via regex. This ensures every query receives a valid `QueryPlan` regardless of API availability.

Downstream influence. The plan drives every subsequent stage through four dictionaries defined at module level:

- `_EXTRACTION_PRIORITIES` — entity type priorities per reasoning type (e.g., causal questions prioritize Cause/Factor and Effect/Outcome nodes);
- `_RELATION_PRIORITIES` — relation type priorities;
- `_ANSWER_STRUCTURES` — output format guidance (e.g., "causal_chain" for causal questions, "table_or_contrast" for comparisons);
- `_MIN_GRAPH_REQUIREMENTS` — minimum node/edge/type counts per reasoning type, used by coverage validation.

3.2 Domain Detector (`domain_detector.py`)

The Domain Detector (Stage 1) classifies the query into one of five supported scientific domains: Agriculture, Geoscience, Computer Science, Supply Chain, or Environment. Three interchangeable strategies are implemented:

1. **LLM strategy** — zero-shot classification via Groq.
2. **Keyword strategy** — rule-based keyword counting against per-domain dictionaries (18–24 keywords each), with LLM fallback when no domain exceeds the match threshold.
3. **Embedding strategy** — cosine similarity between the query embedding and per-domain reference sentence embeddings, computed via the shared `EmbeddingService` singleton.

The production path uses a robust consensus method (`classify_robust()`) that runs keyword and embedding strategies in parallel and uses agreement to select the final domain. Subdomain classification (e.g., "crop science" within Agriculture) is also performed for finer retrieval anchoring.

3.3 Paper Retriever (`paper_retriever.py`)

Stage 2 fetches candidate papers from three external academic APIs:

- **Europe PMC** — free, no authentication, strong coverage of life sciences and agriculture;
- **arXiv** — preprint server, strong for computer science and physics;
- **OpenAlex** — broad interdisciplinary coverage, used as fallback.

Dual retrieval. The retriever issues two parallel queries per source: the original domain-anchored query and an LLM-expanded variant. Query expansion is performed by sending the original query to the Groq LLM with a prompt that requests alternative phrasing and additional search terms. Results from both passes are deduplicated by paper ID.

Plan-aware reranking. After deduplication, papers are reranked using a weighted combination of semantic similarity to the query ($\alpha = 0.6$), keyword overlap with plan entities ($\beta = 0.25$), and metadata quality signals such as citation count and recency ($\gamma = 0.15$). These weights are configurable via `config/settings.py`.

Retry logic. Each API call uses a dedicated retry loop with up to three attempts and configurable backoff delay. The HTTP session is configured with `urllib3.util.Retry` for automatic retries on transient HTTP errors (429, 500, 502, 503).

Concurrency. Source fetches are parallelized using `ThreadPoolExecutor` with `as_completed`, so Europe PMC and arXiv requests execute concurrently rather than sequentially.

3.4 Paper Filter (`paper_filter.py`)

Stage 3 applies a five-step filtering and ranking pipeline:

1. **Abstract filter** — removes papers with missing or short (< 50 characters) abstracts.
2. **Date filter** — removes papers older than a configurable minimum year.
3. **Relevance ranking** — computes a hybrid relevance score combining semantic similarity (weight 0.58), lexical keyword coverage (0.18), metadata credibility prior (0.14), and intent alignment (0.10). Semantic similarity is computed via cosine distance between the query embedding and paper title+abstract embeddings using the shared `EmbeddingService`.
4. **Adaptive percentile filter** — removes papers below the 60th percentile of the relevance score distribution. This is purely embedding-driven with no domain-specific keywords.
5. **Truncation** — returns the top-k papers (default 15, configurable via `TOP_N_PAPERS`).

An evidence consistency assessment (`assess_evidence_consistency()`) computes pairwise lexical overlap among filtered papers and a citation credibility signal, producing a consistency score used downstream to trigger a strict relevance gate when papers disagree.

3.5 Entity Extractor (`entity_extractor.py`)

Stage 4 extracts typed entities and directed relations from each paper's abstract using an LLM. The extraction vocabulary is domain-agnostic:

- **Entity types**: Concept/Entity, Problem/Condition, Method/Intervention, Context/Location, Cause/Factor, Effect/Outcome.
- **Relation types**: affects, mitigates, occurs_in, caused_by, leads_to, detected_by, studied_in, correlates_with, depends_on, part_of.

Three-tier LLM fallback. Extraction attempts follow a cascading strategy:

1. **Full prompt** with structured JSON schema and examples.
2. **Strict JSON retry** with a simplified prompt and explicit format constraints.
3. **Heuristic fallback** using regex-based noun phrase extraction and sentence-level co-occurrence for relations.

Groq rate-limit circuit breaker. When the Groq API returns a 429 (rate limit) error, the extractor sets an internal `_groq_rate_limited` flag and redirects all subsequent extraction calls to the local Ollama endpoint for the remainder of the pipeline run. This avoids cascading failures during bulk extraction.

Concurrent extraction. Papers are processed concurrently using `ThreadPoolExecutor` with a configurable maximum of workers (default 4, controlled by `ENTITY_EXTRACTION_MAX_WORKERS`). A threading lock protects the shared Groq client state.

3.6 Graph Builder (`graph_builder.py`)

Stage 5 constructs a fresh `nx.DiGraph` for each query. The graph is never stored as an instance attribute; it is created inside `build()` and returned explicitly, enforcing per-query isolation.

Entity normalization. Before adding nodes, entities undergo a two-pass normalization:

1. **Canonical text normalization** — lowercasing, punctuation stripping, acronym expansion via a small alias map, and token-signature deduplication.
2. **Embedding similarity clustering** — a union-find structure groups entities of the same type whose embedding cosine similarity exceeds a threshold (default 0.82). The representative entity with the shortest label is kept.

Query-aware filtering. Entities are filtered against the query using a combined semantic (0.7) and lexical (0.3) score. Entities below an adaptive threshold (base 0.30) are removed, unless doing so would leave fewer than five nodes.

Graph densification. After initial construction, the builder applies three optimization passes:

1. **Co-occurrence edges** — entities that co-occur in the same paper abstract at least twice receive an undirected "co_occurs_with" edge.
2. **Semantic bridge edges** — for each node, the top-2 most similar other nodes (above cosine similarity 0.62) receive a "semantically_related" edge, connecting otherwise disconnected components.
3. **Isolated node pruning** — nodes with degree zero that are not query-focused are removed, but only when the graph has more than 12 nodes to avoid over-pruning sparse graphs.

3.7 Graph Retriever (`graph_retriever.py`)

Stage 6 extracts the query-relevant subgraph that will be serialized into the LLM prompt.

Seed selection. Seed entities are selected using a hybrid score:

$$\text{score}_i = 0.72 \cdot \text{sim}_i + 0.28 \cdot \text{importance}_i - 0.20 \cdot \text{hub_penalty}_i$$

where sim_i is the cosine similarity between the query embedding and node i's label embedding, importance_i is a normalized degree centrality, and hub_penalty_i penalizes generic

high-degree nodes that connect to many unrelated concepts. The top-5 seeds above a minimum similarity threshold of 0.35 are selected.

Subgraph expansion. Seeds are expanded via BFS to a depth of 2 hops using the `GraphBuilder.get_subgraph()` method.

Multi-hop reasoning paths. The retriever extracts and ranks reasoning paths — sequences of connected triples that form causal or explanatory chains. Paths are scored by their average edge weight and semantic relevance to the query, and the top-5 paths are included in the LLM context.

Subgraph validation. The validation step checks whether the subgraph meets the minimum requirements specified by the query plan (minimum nodes, edges, and entity type diversity). If validation fails, the retrieval result is flagged with `needs_focus_expansion = True`, which triggers the adaptive expansion loop in `main.py`.

3.8 Adaptive Retry (`adaptive_retry.py`)

The adaptive retry module implements the confidence-driven expansion loop that re-enters the pipeline when graph coverage is insufficient. It provides four functions:

1. `expand_query_with_context()` — analyzes missing requirements (e.g., "Need ≥3 method nodes; found 1") and appends descriptive keywords to the query targeting the missing entity types.
2. `should_retry()` — checks whether the current graph confidence is below the threshold and expansion iterations remain.
3. `merge_graphs()` — merges a newly built graph into the existing one, combining nodes and edges while preserving attributes.
4. `format_retry_summary()` — produces a human-readable summary of what changed during expansion.

The expansion loop in `main.py` is bounded to a maximum of 2 iterations (`MAX_GRAPH_EXPANSION_ITERS`). A set of seen refinement queries prevents infinite loops when the same expanded query is generated twice.

3.9 Answer Generator (`answer_generator.py`)

Stage 7 assembles a RAG-style prompt from the graph triples, reasoning paths, and paper abstracts, then calls the Groq LLM to produce a cited answer.

Sparse graph supplementation. When the retrieved subgraph has fewer edges than `SPARSE_GRAPH_THRESHOLD` (default 3), the top-3 paper abstracts are appended directly to the context text to ensure the LLM has sufficient grounding material.

Intent-specific instructions. The prompt includes intent-aware generation instructions derived from the query plan: comparison questions request tabular contrasts, causal questions request chain explanations, and optimization questions request ranked strategies.

Quality detection and auto-repair. After generation, the answer is checked against quality heuristics (e.g., minimum length, presence of citations, coverage of key entities). If issues are detected, a repair prompt is sent to the LLM requesting targeted improvements.

Rate-limit fallback. If the Groq API returns a rate-limit error during answer generation, the system produces a deterministic fallback answer assembled from paper abstracts and reasoning paths, avoiding a complete pipeline failure.

3.10 Answer Compiler (`answer_compiler.py`)

The Answer Compiler (Stage 7b) is a graph-only answer construction module that does not use the LLM. It ranks graph nodes by a composite score combining degree centrality, query alignment, and paper support, then builds natural-language explanations from graph relations. Its primary role in the production pipeline is as a *quality validator*: the `CompileResult` returned by `compile_answer()` includes coverage and confidence scores, a list of missing requirements, and a quality assessment ("good"/"acceptable"/"insufficient") that the main orchestrator uses to decide whether to trigger adaptive expansion.

3.11 Confidence Scorer (`confidence_scorer.py`)

The Confidence Scorer computes a composite confidence score from five signals:

1. Graph density (from `nx.density()`);
2. Average node degree, normalized as $d/(d+2)$;
3. Edge count, normalized as $\min(E/20, 1)$;
4. Average seed similarity score;
5. Paper support count and reasoning path count.

The overall confidence is a weighted sum:

$$C = 0.25 \cdot \text{density} + 0.25 \cdot \text{degree_norm} + 0.20 \cdot \text{edge_norm} + 0.30 \cdot \text{seed_norm}$$

A coverage score is also computed based on how well the graph satisfies the plan's entity type and relation type requirements.

3.12 Session State (`session_state.py`)

The `SessionState` dataclass manages per-session caching for the Streamlit dashboard, enabling efficient handling of follow-up queries.

- **Graph reuse** — if the new query's embedding is sufficiently similar to the previous query (cosine similarity ≥ 0.82), the existing graph is incrementally updated rather than rebuilt from scratch.
- **Retrieval skip** — if similarity exceeds 0.93, paper retrieval is skipped entirely and the existing paper cache is reused.
- **Bounded caches** — papers and extraction results are stored in `OrderedDict` instances with maximum sizes of 500 and 300 entries respectively, using FIFO eviction.
- **TTL expiration** — all cached data expires after 1800 seconds (30 minutes) of inactivity.

4 Key Design Decisions

4.1 Dynamic Graphs vs. Static Knowledge Bases

The graph is constructed fresh for every query and discarded after answer generation. No persistent graph database is used (the `neo4j` dependency was identified as unused and removed during the final cleanup). This design avoids stale knowledge, ensures that every answer reflects the latest literature, and eliminates the operational complexity of maintaining a graph database. The tradeoff is increased latency per query, which is mitigated by concurrent API fetches and parallel entity extraction.

4.2 Domain-Agnostic Entity Vocabulary

Rather than defining domain-specific entity types (e.g., "Crop", "Mineral"), the system uses six broad categories (Concept/Entity, Problem/Condition, Method/Intervention, Context/Location, Cause/Factor, Effect/Outcome) that apply across all five supported domains. This eliminates the need for per-domain extraction prompts and ensures that adding a new domain requires only updating the keyword dictionary in the Domain Detector.

4.3 Confidence-Driven Adaptive Expansion

The pipeline does not assume that the first retrieval pass will yield sufficient evidence. Instead, the confidence scorer and coverage validator assess the graph after Stage 6, and the adaptive expansion loop re-retrieves and re-builds when coverage is insufficient. This is bounded to 2 iterations to prevent runaway latency, and a deduplication set on refinement queries prevents oscillation.

4.4 Caching Strategy

The `EmbeddingService` singleton maintains a 2000-entry LRU cache of text embeddings, preventing redundant model inference when the same entity label appears across multiple papers. Session-level caching in `SessionState` enables follow-up queries to reuse graphs and papers, reducing latency by an order of magnitude for conversational interactions.

5 Implementation Details

5.1 Libraries and Dependencies

Library	Purpose
`groq`	Groq API client for LLM calls
`openai`	OpenAI-compatible API client
`sentence-transformers`	`all-MiniLM-L6-v2` embedding model
`networkx`	In-memory directed graph construction
`requests`	HTTP client for academic API calls
`scikit-learn`	Cosine similarity computation
`numpy`	Numerical operations on embeddings
`streamlit`	Web dashboard
`pyvis`	Interactive graph visualization
`python-dotenv`	Environment variable management

Table 1: Production dependencies after cleanup.

5.2 External APIs

- **Groq API** — primary LLM backend, using configurable models (default: `llama-3.1-8b-instant` Used for query planning, domain detection, query expansion, entity extraction, and answer generation.
- **Ollama** — local LLM fallback. Each module that calls Groq can optionally route through a local Ollama instance first (`USE_OLLAMA_PRIMARY=1`), with automatic fallback to Groq if Ollama is unavailable.
- **Europe PMC API** — free, no authentication required.
- **arXiv API** — free, XML response format parsed with `xml.etree.ElementTree`.
- **OpenAlex API** — free, JSON response format.

5.3 Concurrency

The system employs concurrency at two levels:

1. **Paper retrieval** — API sources are queried in parallel via `ThreadPoolExecutor` with `as_completed()`. The dual retrieval pattern (original + expanded query) doubles the effective parallelism.
2. **Entity extraction** — paper abstracts are processed concurrently with up to `ENTITY_EXTRACTION_` (default 4) threads. A threading `Lock` protects the Groq client and the rate-limit flag.

5.4 Caching Mechanisms

- **Embedding cache**: `OrderedDict`-based LRU cache in `EmbeddingService`, max 2000 entries.
- **Paper cache**: `OrderedDict` in `SessionState`, max 500 entries, FIFO eviction.
- **Extraction cache**: `OrderedDict` in `SessionState`, max 300 entries.

- **Model health cache**: process-level dictionary in `main.py` that records whether each extractor model passed a precheck, avoiding repeated health probes.

6 Challenges Faced

This section describes real engineering challenges encountered during development, as evidenced by the specific patterns and fallback mechanisms implemented in the codebase.

6.1 API Rate Limiting and Transient Failures

The Groq API imposes per-minute rate limits that become binding during bulk entity extraction, where 10–15 sequential LLM calls are made in rapid succession. The system addresses this at multiple levels:

- Per-API retry loops with exponential backoff (base 0.5s, jitter up to 0.3s) in the entity extractor.
- A circuit-breaker pattern in `EntityExtractor`: when a 429 error is detected, the `_groq_rate_limited` flag is set and all subsequent calls redirect to Ollama for the remainder of the run.
- `urllib3.util.Retry` on the HTTP session in `PaperRetriever` for automatic retries on 429/5xx responses from academic APIs.

6.2 LLM JSON Parsing Failures

The entity extraction prompt requests structured JSON, but LLMs frequently return malformed responses: trailing commas, unquoted keys, markdown code fences wrapping the JSON, or free-text preambles. The three-tier fallback in `EntityExtractor` addresses this progressively:

1. The full prompt attempt parses the response with `json.loads()`.
2. On failure, a regex strips markdown fences and trailing commas before retrying `json.loads()`.
3. If JSON parsing fails entirely, the heuristic fallback extracts entities via noun-phrase regex and infers relations from sentence-level co-occurrence.

6.3 Entity Duplication Across Papers

Multiple papers frequently describe the same concept with different surface forms (e.g., "machine learning", "ML", "Machine-Learning"). Without normalization, the graph becomes cluttered with near-duplicate nodes that dilute centrality scores and reasoning path quality. The two-pass normalization in `GraphBuilder` (canonical text normalization followed by embedding-similarity clustering with union-find) reduces duplication by merging entities above 0.82 cosine similarity within the same type.

6.4 Latency from Sequential API Calls

The initial implementation made sequential HTTP requests to each academic API source, resulting in retrieval times of 60–70 seconds. Parallelizing the API calls via `ThreadPoolExecutor` reduced retrieval latency to approximately 10 seconds, a 6–7× improvement.

6.5 Sparse Graphs for Niche Queries

Queries on narrow topics sometimes yield only 2–3 relevant papers, producing graphs with very few nodes and edges. The system handles this through:

- Sparse graph detection in `AnswerGenerator` (supplementing with raw abstracts when edges < `SPARSE_GRAPH_THRESHOLD`).
- The adaptive expansion loop, which re-retrieves with an expanded query.
- Minimum-node guarantees in the relevance gate (always keeping at least 5 entities).

7 Mistakes and Lessons Learned

7.1 Over-Reliance on LLM Output Structure

Early versions assumed the extraction LLM would consistently return valid JSON. In practice, approximately 15–20% of extraction calls returned malformed output. The lesson was to treat LLM output as untrusted input and always implement parsing with fallback tiers. The three-tier extraction cascade and the heuristic fallback were added as a direct result.

7.2 Hardcoded Configuration Drift

Model names, temperature values, and token budgets were initially hardcoded in individual module constructors. As the number of LLM-calling modules grew to six, these values drifted out of sync, making it impossible to change the model globally without editing every file. Centralizing all configuration in `config/settings.py` with environment variable overrides resolved this.

7.3 Aggressive Filtering Removing Good Papers

An early version of the adaptive percentile filter used a fixed similarity threshold rather than a percentile-based one. This caused queries with uniformly moderate similarity scores to lose all papers. Switching to a percentile-based threshold (keep top 60%) ensured that filtering adapts to the score distribution rather than applying an absolute cutoff.

7.4 Graph Density vs. Noise Tradeoff

The graph densification steps (co-occurrence edges, semantic bridges) were added to improve connectivity, but initially they connected too many unrelated nodes. Tuning the minimum co-occurrence count to 2 and the semantic bridge similarity threshold to 0.62, and skipping generic-node removal on graphs with fewer than 12 nodes, achieved a balance between density and precision.

8 Final Pre-Submission Improvements

The following mechanical fixes were applied during the final cleanup pass, with no changes to pipeline logic or architecture.

8.1 Removed Unused Dependencies

Three packages (`neo4j`, `pandas`, `matplotlib`) were listed in `requirements.txt` but never imported anywhere in the codebase. They were removed to reduce install size and eliminate potential dependency conflicts.

8.2 Fixed Hardcoded Model Names

The string `"llama-3.1-8b-instant"` was hardcoded in six module constructors and one inline Groq call, despite `config/settings.py` providing centralized constants (`QUERY_PLANNER_MODEL`, `ENTITY_EXTRACTION_MODEL`). All occurrences were replaced with the appropriate settings constant, ensuring a single source of truth.

8.3 Fixed `max_tokens` Inconsistency

The `AnswerGenerator._call_llm()` method hardcoded `max_tokens=1024` while `settings.py` defined `ANSWER_PROMPT_TOKEN_BUDGET = 2400` for the answer generation context. The hardcoded value was replaced with the settings constant to prevent silent answer truncation.

8.4 Fixed Temperature Inconsistency

The `QueryPlanner._call_llm()` method hardcoded `temperature=0.0` despite `settings.py` defining `QUERY_PLANNER_TEMPERATURE = 0.1`. The hardcoded value was replaced with the settings constant.

8.5 Logging Improvements

Thirty-five f-string logger calls across `main.py`, `answer_generator.py`, and `paper_retriever.py` were converted to lazy %-formatting (e.g., `logger.info("Found %d papers", count)`). This avoids string interpolation when the log level is disabled and follows Python logging best practices.

8.6 Dead Code Removal

An unused variable `entity_by_label` in `entity_extractor.py` (line 719) was built as a dictionary comprehension but never referenced. It was removed.

9 Evaluation

9.1 Evaluation Framework

The evaluation module (`evaluation/evaluator.py`) implements a four-mode ablation study across a 22-question benchmark dataset:

- 12 single-hop questions (3 per domain: Agriculture, Geoscience, Environment, Supply Chain);
- 10 multi-hop questions requiring cross-paper reasoning.

9.2 Ablation Modes

Each question is answered under four configurations of increasing complexity:

1. **classic_rag** — top-3 paper abstracts concatenated as plain text (no graph, no reasoning, no expansion).
2. **graphrag_base** — full graph pipeline but with reasoning controller, confidence scoring, and expansion disabled.
3. **graphrag_reasoning_only** — graph pipeline with reasoning paths enabled but no confidence-driven expansion.
4. **graphrag_full** — complete pipeline with all features enabled, including adaptive expansion.

9.3 Metrics

The following metrics are computed for each answer:

- **ROUGE-1 F1** — unigram overlap with the reference answer.
- **ROUGE-2 F1** — bigram overlap with the reference answer.
- **Keyword coverage** — fraction of reference keywords found in the generated answer.
- **Semantic similarity** — cosine similarity between the answer embedding and the reference embedding.
- **Citation grounding** — whether the answer cites paper evidence.
- **LLM-as-judge** — a separate LLM call scores the answer on relevance, completeness, and factual accuracy (repeated with configurable judge repeats for stability).

10 Limitations

1. **LLM dependency** — the system relies on external LLM APIs (Groq) for five of its eight stages. API downtime, rate limits, or model deprecation directly impact pipeline availability. The Ollama fallback mitigates but does not eliminate this risk.
2. **Abstract-only extraction** — entities and relations are extracted from paper abstracts only, not full texts. This limits the depth of extracted knowledge, particularly for method-heavy papers where key details appear in the body.
3. **No persistent knowledge accumulation** — the graph is discarded after each query. Repeated questions about the same topic rebuild the graph from scratch (unless session caching applies).
4. **Five-domain scope** — while the entity vocabulary is domain-agnostic, the domain detector and keyword dictionaries currently cover only five scientific domains. Queries outside these domains will be misclassified.
5. **No citation verification** — the system includes paper references in answers but does not verify that the specific claims attributed to a paper are actually present in its abstract.
6. **Single-model extraction** — all extraction uses the same LLM model. A smaller, faster model for simple entities and a larger model for complex relations could improve both speed and quality.
7. **Latency** — end-to-end latency for a full pipeline run (including API calls, extraction, and graph construction) ranges from 15 to 45 seconds depending on query complexity and API response times.

11 Future Work

The following improvements are practical next steps based on observed limitations in the current system:

1. **Full-text extraction** — extend entity extraction beyond abstracts to full paper text (where available), significantly increasing the density and accuracy of extracted knowledge.
2. **Persistent graph accumulation** — maintain a session-level or user-level persistent graph that grows across queries, reducing redundant extraction for related questions.
3. **Retrieval source expansion** — add Semantic Scholar and PubMed as additional retrieval sources for broader coverage.
4. **Extraction model tiering** — use a fast model for initial extraction and a larger model for repair/refinement of low-confidence extractions.
5. **Citation grounding verification** — cross-reference specific claims in the generated answer against the source paper abstracts to detect hallucinated citations.
6. **Domain auto-detection expansion** — allow the system to handle queries outside the five predefined domains by falling back to a fully LLM-driven classification without keyword dictionaries.
7. **Streaming answers** — implement token-level streaming from the LLM API to the Streamlit frontend, reducing perceived latency.
8. **Evaluation expansion** — increase the benchmark dataset beyond 22 questions and add domain-expert human evaluation alongside automated metrics.

12 Conclusion

Dynamic GraphRAG demonstrates that constructing ephemeral knowledge graphs from live academic literature meaningfully improves scientific question answering compared to flat retrieval-augmented generation. The system's key contributions are:

1. A plan-driven pipeline architecture where an LLM-generated query plan shapes every stage from retrieval weighting to answer structure.
2. A domain-agnostic entity and relation vocabulary that supports five scientific domains without per-domain prompts or keyword lists.
3. A confidence-driven adaptive expansion loop that automatically re-retrieves and rebuilds the graph when coverage is insufficient, bounded to prevent runaway latency.
4. A three-tier extraction fallback (full prompt, strict JSON, heuristic) with a Groq-to-Ollama circuit breaker that ensures extraction completes even under API rate limits.
5. A four-mode ablation evaluation framework that isolates the contribution of reasoning, confidence scoring, and adaptive expansion.

The system is fully operational, with a Streamlit dashboard providing interactive querying, knowledge graph visualization, paper browsing, and pipeline debugging. All configuration is centralized in `config/settings.py` with environment variable overrides, and the codebase has been cleaned for production readiness.